

THE CONSTITUTION FOR ACADEMIC PYTHON PROJECTS

Portfolio Edition

A governance standard for authorship, scope, design quality, verification, and reflective engineering practice.

Adopted for use in the evaluation and documentation of academic Python software projects.

Preamble

This Constitution establishes the governing standard for the evaluation of academic Python projects produced under this roadmap. Its purpose is to preserve authorship integrity, enforce proportionate scope discipline, reward sound engineering judgment, and document real progression toward software and system design competence. It shall be construed reasonably, with due regard for project size, stated intent, and learner stage. It is not designed to punish good-faith work. It is designed to ensure that work offered as evidence is honest, intentional, understandable, verifiable, and professionally defensible.

Article 1 — Python Fundamentals and Architectural Judgment

Section 1.1

A project shall demonstrate Python fundamentals and architectural thinking appropriate to its scope. Such proof may consist of clear structure, understandable control flow, sensible decomposition, deliberate boundaries, and technical choices proportionate to the project's size and purpose.

Amendment 1.1 — Dual Evaluation for Grouped Applications

Where an application is intentionally designed as a component within a grouped system, it shall receive both (a) a standalone verdict and (b) a component verdict. Incompleteness in isolation shall not, by itself, constitute failure where the component verdict confirms intentional design and fit within the larger system.

Amendment 1.2 — SysForge Monorepo DRY Structure

The SysForge monorepo DRY structure shall be treated as an architectural feature rather than as scope creep, provided that shared utilities, common patterns, and reuse are intentional and materially improve consistency, maintainability, or composition.

Amendment 1.3 — Shared Module Policy

Copied shared modules are not the preferred standard. The accepted standard is use of PYTHONPATH or a pinned snapshot. Where copied shared modules are temporarily used, such usage shall be flagged, resolved before final documentation, recorded in Lessons Learned, and accompanied by notation that a shared core package is the preferred long-term solution.

Article 2 — Honest Skill Representation and Authorship

Section 2.1

A project shall honestly reflect the developer's own skill, effort, and reasoning. Work shall not be presented as independent authorship where core logic, architecture, or problem decomposition was materially outsourced.

Amendment 2.1 — Permitted Reference Use; Flagged AI Use

The following shall not constitute violations: documentation-sourced solutions, Stack Overflow research, reference patterns for specific features, known regex patterns, and AI assistance used solely to comment or explain developer-authored code. The following shall be flagged: AI-authored logic, AI-authored architecture, AI-authored decomposition, and AI-generated implementations presented as original work. A

flag does not automatically invalidate a project, but it diminishes the project's force as evidence of independent ability and must be disclosed.

Article 3 — Scope Discipline

Section 3.1

A project shall remain appropriately scoped for its intended learning objective. Scope discipline is itself a core engineering skill and shall be evaluated alongside functionality and design.

Section 3.2

Unless expressly classified otherwise, the default standard is that a project should be completable in twenty-four hours or less.

Amendment 3.1 — Small Multi-File Exception

A multi-file project is permitted where the files are small and single-responsibility. Automatic exceptions apply to error handling modules, CLI parsing modules, `__init__.py`, and `__main__.py`, provided such files support structure rather than needless complexity.

Amendment 3.2 — Formatting and Rendering Modules

Dedicated output formatting or rendering modules, including `formatter.py`, `renderer.py`, and `display.py`, are exempt where they remain under one hundred lines, retain a single responsibility, and do not create unnecessary architectural sprawl.

Amendment 3.3 — Data and Constants Modules

Dedicated static data modules, including `common_passwords.py`, `constants.py`, and `config_defaults.py`, are exempt where they contain only static data and include no logic, functions, or classes.

Amendment 3.4 — Larger Project Classifications

Where the roadmap advances beyond small CLI exercises, the following expanded scope standards shall apply: mini-projects, twenty-four hours or less; medium projects, two to five days; flagship projects, one to two weeks. Larger scopes are permissible only where justified by project type and objective.

Article 4 — Engineering Quality

Section 4.1

A valid project shall demonstrate engineering quality appropriate to its size. Enterprise-level complexity is not required; however, care in structure, naming, decomposition, boundaries, and failure handling is required.

Section 4.2

Engineering quality shall be assessed by reference to separation of concerns, predictable control flow, readable naming, appropriate modularity, minimal unnecessary duplication, sensible input and output boundaries, understandable error handling, and maintainable structure.

Amendment 4.1 — Quality Relative to Scope

Quality shall be judged relative to the project’s purpose and scale. A small CLI utility shall not be penalized for lacking the structure of a larger system; a larger project, by contrast, shall be expected to demonstrate stronger boundaries and clearer organization.

Amendment 4.2 — Cleverness Not Controlling

Dense, compressed, or clever code shall not be presumed superior. Readability, maintainability, and clarity shall take precedence over novelty.

Article 5 — Trade-Offs and Constraints

Section 5.1

Each project shall include some record of trade-offs and constraints. The purpose of this requirement is to cultivate architectural judgment rather than mere implementation output.

Section 5.2

Each project should identify, at minimum, the reason the chosen design was adopted, what was intentionally omitted, the project’s principal weakness, what would change if the system required scale, and the most likely next refactor.

Amendment 5.1 — Short Reflection Suffices

A short reflection shall suffice. The standard is clarity rather than length. Five to eight thoughtful bullet points may satisfy this requirement if they demonstrate genuine design awareness.

Amendment 5.2 — Intentional Omission Permitted

A project need not solve every conceivable problem. Omission is permitted where it is intentional, proportionate, and acknowledged.

Article 6 — Verification of Behavior

Section 6.1

A project shall provide evidence that its behavior was verified. Mere assertion that the project runs locally shall not, standing alone, satisfy this requirement.

Section 6.2

At least one of the following shall be present: a manual test checklist, sample input and output cases, unit tests for core logic, edge-case validation notes, or a demonstration transcript. The chosen method shall be proportionate to project size and complexity.

Amendment 6.1 — Proportional Verification

Small projects may rely on manual verification or sample runs. Larger projects should increasingly employ repeatable validation methods, including unit tests, scenario tests, or structured test notes.

Amendment 6.2 — Priority of Core Logic

Where time is limited, verification shall prioritize core logic over wrappers, cosmetics, or display formatting.

Article 7 — Progressive Complexity

Section 7.1

Projects shall be evaluated not only individually but also collectively as a body of work. The portfolio shall evidence growth over time.

Section 7.2

Across the roadmap, the work should progressively demonstrate advancement in control flow, functions and modularity, data structures, file I/O, parsing and validation, error handling, state management, persistence, APIs, concurrency, and system design thinking.

Amendment 7.1 — Repetition by Design

Repetition shall not be treated as a defect where it is intentional and used to reinforce skills, compare approaches, or apply the same concept under different constraints.

Amendment 7.2 — Progression Over Raw Count

The number of completed applications shall carry weight only where accompanied by visible increases in sophistication, judgment, or design quality.

Article 8 — Final Evaluation Standard

Section 8.1

A project shall be deemed valid under this Constitution where it (a) demonstrates authentic developer-authored work, (b) remains appropriately scoped, (c) exhibits sound architectural thinking for its size, (d) demonstrates acceptable engineering quality, (e) includes some form of behavior verification, and (f) contributes to the broader progression of the roadmap.

Section 8.2

A project need not be perfect to pass. Imperfection is expected in learner work. It must, however, remain honest, intentional, understandable, verifiable, and reflective.

Amendment 8.1 — Imperfection Not Failure

A project may pass despite identifiable weaknesses, provided such weaknesses are acknowledged and do not materially undermine the project's core learning value.

Amendment 8.2 — Flags Not Necessarily Disqualifying

A flagged issue does not automatically invalidate a project. Some flags indicate improvement opportunities rather than disqualifying defects. Final judgment shall consider severity, candor, and overall alignment with this Constitution.

Evaluation Axes

- Authorship Integrity — Whether the work is genuinely developer-authored.
- Scope Discipline — Whether the project is appropriately sized and controlled.
- Architectural Structure — Whether the project shows sound boundaries, decomposition, and organization.
- Behavior Verification — Whether there is credible evidence that the system was tested or validated.
- Reflection and Trade-Offs — Whether the project demonstrates judgment regarding constraints, omissions, and future improvements.

Verdict Labels

- Pass
- Pass with Flags
- Component Pass
- Needs Revision
- Invalid as Evidence of Independent Ability

Construction Clause

This Constitution shall be applied in good faith. Its purpose is to create a rigorous yet fair standard for evaluating academic engineering work. Where ambiguity exists, interpretation should favor honesty, proportionality, and the educational value of the work, while preserving the integrity of the portfolio as evidence of genuine competence.